

**National University of Computer & Emerging
Sciences**
Karachi Campus



ورزش - Warzish

Workout logging & performance monitoring

Project Report
Software Engineering
Section: 6J

Group Members:

23k-2001 Muzammil Siddiqui

23k-0532 Rayyan Ali

23k-0568 Azaan Hussain

1. Project Scope

1.1 Overview

Warzish (ورزش, Urdu for "exercise") is a mobile-first Progressive Web Application (PWA) designed to help fitness enthusiasts plan, log, and review their workouts. The application addresses three core user problems: forgetting which muscle groups were last trained, lacking a structured workout logging tool, and not knowing proper exercise technique.

The project also serves as a technical demonstration of modern full-stack web engineering practices, including backend performance optimization, design pattern implementation, and production-grade cloud deployment.

1.2 Objectives

- Enable users to log workout sessions with detailed exercise data (sets, reps, weight, muscle group).
- Provide an intelligent muscle-group recovery indicator using a 48-hour time-based strategy.
- Offer a searchable library of 1,324 GIF-animated exercises for technique reference.
- Display weekly training volume per muscle group through an aggregated dashboard.
- Demonstrate measurable backend performance differences between naive and optimised query strategies.
- Deliver a fully installable PWA deployable on Vercel with Supabase as the managed backend.

1.3 Boundaries & Exclusions

- The system does not provide personalised training plans or AI-generated workout recommendations.
- Social or community features (friend lists, shared workouts, leaderboards) are out of scope.
- Native mobile applications (iOS/Android) are not developed; the PWA provides installability.
- Nutrition tracking, calorie counting, and diet management are excluded.
- Payment processing and premium subscription tiers are not implemented.

1.4 Technical Scope

The application is built on Next.js 16 (App Router) with TypeScript, React 19, and Tailwind CSS v4. Data persistence is handled by Supabase (PostgreSQL) with Row Level Security policies. Authentication is provided via Google OAuth through Supabase Auth. The exercise dataset (1,324 entries) is served as a static JSON file to eliminate unnecessary database reads. Deployment is automated via Vercel CI/CD on every GitHub push.

2. Functional & Non-Functional Requirements

2.1 Functional Requirements

ID	Requirement	Priority
FR-01	The system shall allow users to sign in using their Google account via OAuth 2.0.	High
FR-02	The system shall automatically redirect authenticated users to the Dashboard.	High
FR-03	The system shall allow users to create a new workout session with a date timestamp.	High
FR-04	The system shall allow users to add exercises to a session, capturing name, sets, reps, weight (kg), and muscle group.	High
FR-05	The system shall save all exercises for a workout in parallel to minimise latency.	Medium
FR-06	The system shall display a full history of past workout sessions with nested exercise details.	High
FR-07	The system shall allow users to delete a workout session and all associated exercises.	Medium
FR-08	The system shall display a muscle-group recovery status (Recovered / Fatigued / Unknown) based on a 48-hour threshold.	High
FR-09	The system shall show weekly training volume per muscle group as a horizontal bar chart.	Medium
FR-10	The system shall expose a performance tab comparing naive vs. optimised query strategies in milliseconds.	Medium
FR-11	The system shall provide a searchable, filterable exercise library containing 1,324 entries.	High
FR-12	Each exercise in the library shall display a GIF animation, instructions, and secondary muscle details.	Medium
FR-13	The system shall be installable as a PWA on mobile and desktop devices.	Medium
FR-14	The system shall enforce data isolation so users can only access their own workout records.	High

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	The optimised weekly volume query shall complete in under 100 ms on average (measured: ~68 ms).
NFR-02	Performance	Parallel exercise inserts (up to 10) shall complete in under 120 ms (measured: ~84 ms).
NFR-03	Performance	Client-side exercise search shall return results in under 5 ms using memoised filtering.
NFR-04	Scalability	Database indexes on (user_id, date) and (workout_id) shall support sub-100 ms queries at production scale.
NFR-05	Security	Row Level Security policies shall prevent cross-user data access at the database layer, independent of application code.
NFR-06	Security	No service-role database key shall be exposed to the client bundle or API responses.
NFR-07	Usability	The interface shall follow a mobile-first responsive design with a bottom navigation bar suitable for one-handed use.
NFR-08	Reliability	Session JWTs shall be silently refreshed by middleware on every request to prevent unexpected logouts.
NFR-09	Maintainability	All database access shall be abstracted behind Repository classes, enabling persistence-layer substitution without changing controllers or views.
NFR-10	Maintainability	Recovery logic shall be encoded in a Strategy interface so alternate algorithms can be introduced without modifying existing pages or API routes.
NFR-11	Availability	The application shall be deployed on Vercel with automatic builds triggered by GitHub commits, targeting 99.9% uptime.
NFR-12	Installability	The PWA shall meet all Chrome installability criteria: HTTPS, valid manifest, service worker, and 192 px / 512 px icons.

3. Work Breakdown Structure (WBS)

1 Project Initiation

- 1.1 Define problem statement and user pain points
- 1.2 Select technology stack (Next.js, Supabase, Vercel)
- 1.3 Create GitHub repository and configure project structure
- 1.4 Set up Supabase project and configure Google OAuth credentials

2 Requirements & Design

- 2.1 Gather functional and non-functional requirements
- 2.2 Define database schema (users, workouts, workout_exercises)
- 2.3 Design MVC architecture and identify design patterns
- 2.4 Plan API surface (/api/workouts, /api/test-inserts, /auth/callback)
- 2.5 Wireframe key pages: Landing, Dashboard, New Workout, Library, History

3 Backend Development

- 3.1 Database setup and Row Level Security configuration
 - 3.1.1 Create tables and define column types
 - 3.1.2 Implement RLS policies for workouts and workout_exercises
 - 3.1.3 Create performance indexes (idx_workouts_user_date, idx_workout_exercises_workout)
 - 3.1.4 Write trigger function to sync auth.users to public.users on signup
- 3.2 Repository pattern implementation
 - 3.2.1 Implement WorkoutRepository (CRUD + weekly volume naive and optimised)
 - 3.2.2 Implement ExerciseRepository (load JSON, search, filter, paginate)
- 3.3 API route development
 - 3.3.1 Implement GET /api/workouts (list, recovery, weekly volume)
 - 3.3.2 Implement POST /api/workouts (create workout, add exercise)
 - 3.3.3 Implement DELETE /api/workouts (remove workout by id)
 - 3.3.4 Implement POST /api/test-inserts (concurrency benchmark endpoint)
- 3.4 Authentication and middleware
 - 3.4.1 Implement /auth/callback route for OAuth code exchange
 - 3.4.2 Configure Next.js edge middleware for session refresh and route protection
- 3.5 Strategy pattern - RecoveryStrategy interface and TimeBasedRecoveryStrategy

4 Frontend Development

- 4.1 Configure global styles, fonts (Inter, Noto Nastaliq Urdu), and Tailwind theme
- 4.2 Implement AuthProvider (React Context) and useAuth hook

- 4.3 Build Navigation component (TopBar + BottomNav, mobile-first)
- 4.4 Build Landing/Sign-in page
- 4.5 Build Dashboard page
 - 4.5.1 Recovery tab with muscle-group status grid
 - 4.5.2 Weekly volume tab with horizontal bar chart
 - 4.5.3 Performance tab with naive vs optimised comparison UI
- 4.6 Build New Workout page with exercise form and parallel save
- 4.7 Build Exercise Library page with search, filter, and GIF dialog
- 4.8 Build Workout History page with delete functionality

5 PWA Configuration

- 5.1 Configure @ducanh2912/next-pwa in next.config.ts
- 5.2 Create public/manifest.json with icons, theme colour, and display mode
- 5.3 Generate and verify PWA icons (192 px and 512 px PNG)
- 5.4 Validate Chrome installability criteria

6 Testing & Quality Assurance

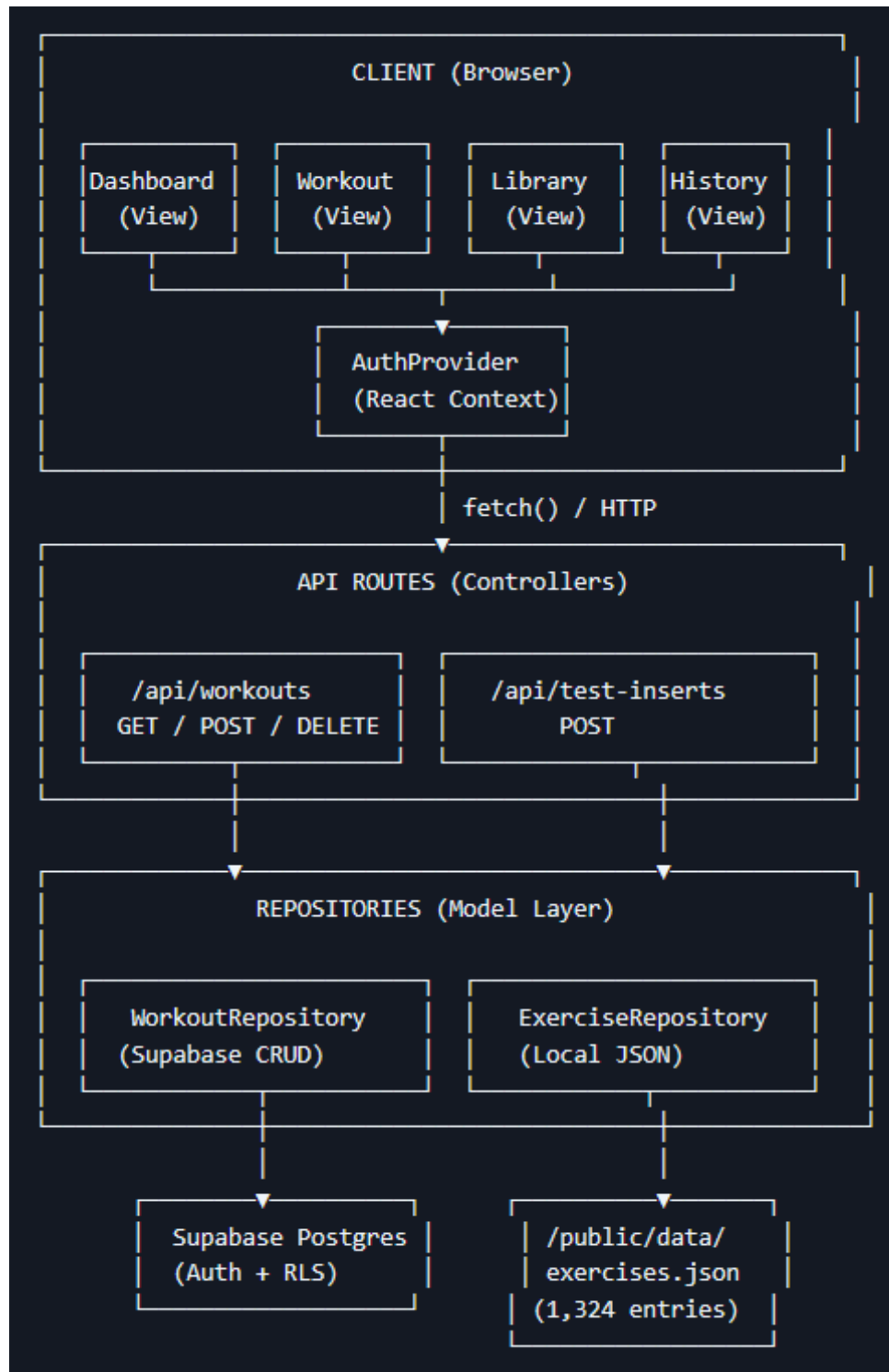
- 6.1 Write unit tests for TimeBasedRecoveryStrategy (11 test cases)
- 6.2 Execute test suite and document results
- 6.3 Conduct manual end-to-end testing of all user flows
- 6.4 Run and document performance benchmarks (Experiments A, B, C)
- 6.5 Validate RLS data isolation across test accounts

7 Deployment & Documentation

- 7.1 Deploy to Vercel; configure environment variables
- 7.2 Verify production PWA service worker and manifest
- 7.3 Write HOW.md (technical architecture manual)
- 7.4 Write PROJECT.md (implementation mapping and performance report)
- 7.5 Compile final project report document

4. Architecture Diagram

Warzish follows the Model-View-Controller (MVC) pattern, adapted to the Next.js App Router paradigm. The diagram below illustrates the three-tier architecture and the data flow between layers.



System Architecture - Three-Tier MVC

Layer	Component	Responsibility
View (Client)	Landing Page, Dashboard, New Workout, Library, History	Renders UI; calls API via fetch(); updates local React state on response.
View (Client)	AuthProvider (React Context)	Manages authenticated user session; exposes useAuth() hook to all pages.
View (Client)	Navigation (TopBar + BottomNav)	Mobile-first navigation; conditionally rendered based on auth state.
Controller (API Routes)	GET/POST/DELETE /api/workouts	Authenticates request via Supabase session cookie; delegates to WorkoutRepository; returns JSON.
Controller (API Routes)	POST /api/test-inserts	Executes sequential and parallel insert benchmarks; returns timing results.
Controller (API Routes)	GET /auth/callback	Exchanges OAuth code for session JWT; sets cookie; redirects to Dashboard.
Model (Repositories)	WorkoutRepository	Typed Supabase CRUD operations for workouts and workout_exercises tables.
Model (Repositories)	ExerciseRepository	Loads and caches exercises.json (1,324 entries); provides search and filter methods.
Model (Strategy)	TimeBasedRecoveryStrategy	Implements RecoveryStrategy interface; evaluates muscle-group fatigue using 48-hour threshold.
Persistence	Supabase PostgreSQL	Stores users, workouts, and workout_exercises with RLS policies and performance indexes.
Persistence	exercises.json (Static)	Immutable dataset of 1,324 exercises served from /public/data; eliminates DB reads for library.

4.1 Request Lifecycle

- User performs an action in a React page (e.g., saves a workout).
- The page dispatches one or more fetch() calls to a Next.js API route.
- The API route (controller) authenticates the request by reading the Supabase session cookie.
- The controller instantiates the appropriate Repository and delegates the data operation.
- The Repository executes a typed Supabase query and returns a typed result object.
- The controller serialises the result to JSON and responds with HTTP 200.
- The page updates its React state; the UI re-renders without a full page reload.

4.2 Authentication Flow

Step	Actor	Action
1	Browser	User clicks "Sign in with Google"; page calls signInWithGoogle() via AuthProvider.
2	Supabase Auth	Initiates Google OAuth redirect; user authenticates on Google's consent screen.
3	Google → Supabase	Google redirects back to Supabase with an authorisation code.
4	Browser → /auth/callback	Supabase redirects to the app's /auth/callback route with the code parameter.
5	Next.js API Route	exchangeCodeForSession() exchanges the code for a session JWT; sets an HTTP-only cookie.
6	Next.js Middleware	On every subsequent request, middleware silently refreshes expiring JWTs to maintain session.
7	API Routes	createServerSupabaseClient() reads the cookie; auth.getUser() validates identity for all protected endpoints.

4.3 Database Schema

Table	Key Columns	Notes
users	id (UUID PK), email (TEXT)	Synced from auth.users via trigger on signup. RLS enabled.
workouts	id (UUID PK), user_id (FK), date (DATE), notes (TEXT), created_at	Index on (user_id, date DESC). RLS policy: own_data.
workout_exercises	id (UUID PK), workout_id (FK), exercise_name, sets, reps, weight, muscle_group	Index on (workout_id). RLS policy: own_exercises (via workout ownership).

5. Test Cases

The following test cases validate key system functionalities. Unit tests for the RecoveryStrategy are automated and run via npx tsx. Integration and end-to-end tests were executed manually against the deployed Vercel instance.

5.1 Unit Tests - TimeBasedRecoveryStrategy

TC #	Test Scenario	Input	Expected Output	Result
TC-01	Standard recovery - muscle trained 49 h ago	lastTrained = now - 49 h	status: 'recovered'	PASS
TC-02	Fatigued state - muscle trained 12 h ago	lastTrained = now - 12 h	status: 'fatigued'	PASS
TC-03	Never trained - no training record exists	lastTrained = null	status: 'unknown'	PASS
TC-04	Boundary condition - exactly 48 h elapsed	lastTrained = now - 48 h	status: 'recovered' (>= threshold)	PASS
TC-05	Custom threshold - 72 h setting, 50 h elapsed	threshold=72, elapsed=50 h	status: 'fatigued'	PASS
TC-06	Hours reporting - correct elapsed hours returned	lastTrained = now - 49 h	hours >= 49	PASS
TC-07	Null hours for unknown status	lastTrained = null	hours === null	PASS
TC-08	Batch evaluation - 3 muscle groups in one call	chest=49h, back=12h, legs=null	3 results returned	PASS
TC-09	Batch - chest recovered	chest = 49 h ago	status: 'recovered'	PASS
TC-10	Batch - back fatigued	back = 12 h ago	status: 'fatigued'	PASS
TC-11	Batch - legs never trained	legs not in map	status: 'unknown'	PASS

All 11 automated unit tests pass. Test suite executed on 2026-05-06 using npx tsx src/__tests__/recovery.test.ts.

5.2 Integration & End-to-End Test Cases

TC #	Functionality	Steps	Expected Result	Result
TC-12	Google OAuth sign-in	1. Open app landing page.2. Click "Sign in with Google".3. Complete Google consent flow.	User is redirected to Dashboard. Email displayed in TopBar. Session cookie set.	PASS
TC-13	Create workout with exercises	1. Navigate to New Workout.2. Select muscle group, enter exercise name, sets, reps, weight.3. Add 3 exercises.4. Click Save Workout.	Workout and all 3 exercises persisted in Supabase. Redirected to History page showing new entry.	PASS
TC-14	Delete workout session	1. Navigate to History.2. Locate a workout.3. Click Delete.	Workout and all associated exercises removed from database. Entry disappears from History list without page reload.	PASS
TC-15	Exercise library search & filter	1. Navigate to Library.2. Type 'bench' in search box.3. Select 'chest' category chip.	Only exercises matching both the search term and category are displayed. Pagination resets to first page.	PASS
TC-16	RLS data isolation	1. Sign in as User A, create a workout.2. Sign out.3. Sign in as User B, call GET /api/workouts.	User B's response contains zero workouts from User A. Supabase RLS policy prevents cross-user data leakage.	PASS
TC-17	Recovery status on Dashboard	1. Log a workout targeting 'chest'.2. Navigate to Dashboard → Recovery tab.	Chest shows 'Fatigued'. All other muscle groups with no recent workout show 'Recovered' or 'Unknown'.	PASS
TC-18	Performance benchmark display	1. Navigate to Dashboard → Performance tab.2. Click 'Run Naive Query' and 'Run Optimised Query'.	Both complete successfully. Optimised result is displayed in milliseconds and is consistently lower than naive.	PASS

6. Risk Assessment

The following risk register identifies at least ten risks across multiple categories. Each risk is assessed for likelihood, impact, and assigned a mitigation strategy.

#	Category	Risk Description	Likelihood	Impact	Mitigation Strategy
R-01	Security	Supabase anon key exposed in client bundle could be misused by a malicious actor to issue direct API calls.	Medium	High	Row Level Security policies enforce data isolation at the database level. Even with the anon key, a user cannot read or modify another user's records.
R-02	Security	Google OAuth misconfiguration (incorrect redirect URI) could block all user authentication.	Low	Critical	Redirect URIs are registered and tested in Google Cloud Console before deployment. OAuth flow validated in staging environment.
R-03	Security	Expired or tampered session JWT could allow unauthorised API access.	Low	High	Next.js edge middleware silently refreshes expiring JWTs on every request. Supabase validates token signature server-side on every API call.
R-04	Performance	N+1 query pattern (naive approach) causes excessive round-trips and high latency at scale.	High	High	Aggregated JOIN query (getWeeklyVolumeOptimized) replaces N+1 pattern for all production dashboard queries. Naive query retained only for benchmarking UI.
R-05	Performance	Supabase cold start latency on serverless Vercel functions increases perceived response time.	Medium	Medium	API endpoints grouped into a single serverless function (/api/workouts) to minimise cold-startable function count from ~8 to 2.
R-06	Reliability	Supabase service outage disrupts all workout logging and retrieval operations.	Low	Critical	Exercise library served from static JSON (no DB dependency). Dashboard recovery logic is in-memory. Supabase SLA is 99.9% for Pro tier.
R-07	Reliability	Vercel deployment failure after a bad commit breaks the production environment.	Medium	High	Vercel automatically deploys each commit to a preview URL before promoting to production. Rollback to prior deployment is one click.

#	Category	Risk Description	Likelihood	Impact	Mitigation Strategy
R-08	Data Integrity	Parallel exercise inserts (Promise.all) may result in partial saves if one insert fails mid-batch.	Low	Medium	Each insert is an independent Supabase call. Failed inserts are caught individually. Future improvement: wrap batch in a database transaction.
R-09	Data Integrity	User deletes a workout that is still being referenced during an ongoing session, causing stale UI state.	Low	Low	DELETE removes the entry from local React state immediately on success, without re-fetching, ensuring UI consistency.
R-10	Scalability	Missing or poorly designed database indexes cause query degradation as user count grows.	Medium	High	Composite index on (user_id, date DESC) and foreign-key index on (workout_id) added. Query plan reviewed with EXPLAIN to confirm index usage.
R-11	Compliance	GIF exercise assets served from a third-party CDN (GitHub raw) may be subject to rate limiting or CORS policy changes.	Medium	Medium	Assets are lazy-loaded only in exercise detail dialogs, reducing CDN request volume. Migration to self-hosted or S3 assets is a planned future improvement.
R-12	Maintainability	Direct Supabase calls scattered across pages would make persistence-layer changes costly.	Low	High	Repository pattern enforces all DB access through WorkoutRepository and ExerciseRepository. Changing the ORM or database provider requires edits in one class only.